

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

Universidad de Sevilla

Escuela Técnica Superior de Ingeniería Informática

Documentación de la entrega M01

Documentación Configuration Report



Grado en Ingeniería Informática – Ingeniería del Software

Proceso y Gestión Software 2

Curso 2023 – 2024

Fecha	Versión
28/02/2024	v1r7

Grupo de prácticas: G5-54		
Autores	Rol	Correo electrónico
Blanco Mora, David	Developer	davblamor@alum.us.es
García Lama, Gonzalo	Scrum Manager	gongarlam@alum.us.es
Martín de Prado Barragán, Carlos	Developer	carmarbar9@alum.us.es
Núñez Sánchez, Juan	Developer	juanunsan2@alum.us.es
Jiménez Soriano, Lidia	Developer	lidjimsor@alum.us.es
Yao, Jun	Developer	junyao@alum.us.es

Repositorio: <https://github.com/gii-is-psg2/psg2-2324-g5-54>

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

Índice de contenido

1. Introducción	2
2. Control de Versiones	3
3. Estándares de codificación	4
4. Políticas de mensajes de commit	5
5. Estructura de los repositorios y ramas predeterminadas	6
6. Estrategia de ramificación, basada en Git Flow e incluyendo revisiones por pares	8
6.1. Cómo desarrollar ramas de características	8
6.2. Cómo preparar lanzamientos	8
6.3. Cómo solucionar errores en producción	9
7. Políticas de versionado	9
8. Definición de “hecho”	9
9. Definición de “Coordinación”	11
10. Cómo se gestionan los documentos generados durante el proyecto	12
11. Objetivos y ámbito de CMDB	13

1. Introducción

En este informe se detallan las directrices y políticas establecidas para la gestión del proyecto, así como los procedimientos y estándares adoptados en áreas clave como el control de versiones, la codificación, la estructura de repositorios y las estrategias de ramificación. Además, se describen las normativas referentes a la definición de tareas concluidas, la gestión de la documentación generada y otras prácticas relevantes para el desarrollo ágil del software.

Este informe, elaborado en colaboración por los autores mencionados, proporciona una visión integral de las políticas y procedimientos adoptados durante el Sprint 1, sentando así las bases para el desarrollo continuo del proyecto en los siguientes ciclos de trabajo.

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

2. Control de Versiones

Fecha	Versión	Descripción	Sprint
20/02/2024	v1r0	Creación del documento	1
21/02/2024	v1r1	Integración de los puntos restantes	1
23/02/2024	v1r2	Completación de información en algunos puntos	1
27/02/2024	v1r3	Actualización de puntos con ejemplos	1
27/02/2024	v1r4	Modificaciones en puntos con capturas	1
27/02/2024	v1r5	Añadido apartado 10 "CMDB"	1
28/02/2024	v1r6	Añadida "Definición de Coordinación"	1
28/02/2024	v1r7	Entrega	1

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

3. Estándares de codificación

Para mantener un código coherente y legible, se siguen las siguientes normas de codificación:

- Para el **nombrado de variables locales**, variables globales y constantes se usarán nombres descriptivos y siguiendo las siguientes reglas:

Variables Locales: Para el nombrado de variables locales siempre las escribiremos en minúscula excepto para la primera letra de cada palabra, exceptuando la primera, es decir, se usará la regla '**camelCase**' (e.j `localVariable`).

Variables Globales: Para el nombrado de variables locales siempre las escribiremos en minúscula excepto para la primera letra de cada palabra (e.j `GlobalVariable`).

Constantes: Para el nombrado de constantes usaremos solo letras mayúsculas (e.j `CONSTANT`)

En la siguiente imagen de nuestro proyecto, en concreto en la clase implementada '**PetHotelController**', se pueden observar varias variables que siguen las normas de codificación:

```

public class PetHotelController {
    @Autowired
    private PetHotelService hotelService;
    @Autowired
    private UserService userService;
    private static final String ADMIN_AUTH = "ADMIN";
    private static final String CLINIC_OWNER_AUTH = "CLINIC_OWNER";

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ResponseEntity<PetHotelRoom> create(@RequestBody @Valid PetHotelRoom hotel) throws DataAccessException {
        User user = userService.findCurrentUser();
        PetHotelRoom newHotel = new PetHotelRoom();
        PetHotelRoom savedHotel;
        BeanUtils.copyProperties(hotel, newHotel, "id");
        if (user.hasAnyAuthority(CLINIC_OWNER_AUTH).equals(true) || user.hasAnyAuthority(ADMIN_AUTH).equals(true)) {
            savedHotel = hotelService.crearHotel(newHotel);
        }
    }
}

```

- **Indentación** consistente en todo el proyecto para mejorar la calidad del código y que sea legible y tenga un formato coherente. La sangría facilita la identificación de fragmentos de código y fortalece la organización general del código.

- **Comentar el código** de manera clara y concisa. Esto además aporta actúan como un recurso útil para otros programadores que luego necesiten agregar o modificar el código.

	PSG2 Documentación de la entrega Milestone 01 Documentación Configuration Report
---	--

- **Evitar la duplicación de código** mediante la reutilización y la modularidad. Esto acelera el desarrollo, asegura la consistencia y reduce la posibilidad de cometer errores.

-**Control de versiones y colaboración** resulta esencial para el desarrollo web moderno. El uso de herramientas de control de versiones como Git facilita la colaboración y el avance del proyecto de forma óptima.

-**Pruebas** ayudan a asegurarse de que se entregue código de alta calidad. Es más fácil de encontrar y solucionar problemas al principio del proceso de desarrollo cuando se utilizan pruebas unitarias y pruebas de integración.

-**Aprendizaje y mejora continua** los desarrolladores debemos adoptar una mentalidad de continua aprendizaje y mejora para producir código de la más alta calidad durante todas las fases de desarrollo.

4. Políticas de mensajes de commit

Los mensajes de commit deberán ser descriptivos y concisos, incluyendo información relevante sobre los cambios realizados. Se siguen las siguientes convenciones:

-Uso de verbos en **modo imperativo** en el título.

-**Prefijar los mensajes** de commit con uno de los siguientes tipos:

- **task:** para las tareas que hemos hecho individualmente.
- **fix:** para correcciones de errores.
- **docs:** para cambios en la documentación.
- **test:** para cuando se añadan tests

-No usar **puntos finales** ni suspensivos al final del título.

-El mensaje de commit no deberá contener ningún mensaje de **espacios en blanco**.

-Ser **corto y conciso** en los títulos, para ello se establecen un máximo de 50 caracteres.

-**Uso de mayúsculas** al inicio del título y por cada párrafo del cuerpo del mensaje.

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

Ejemplos de commits realizados durante este Sprint con la política explicada anteriormente:

Tarea 1.6a

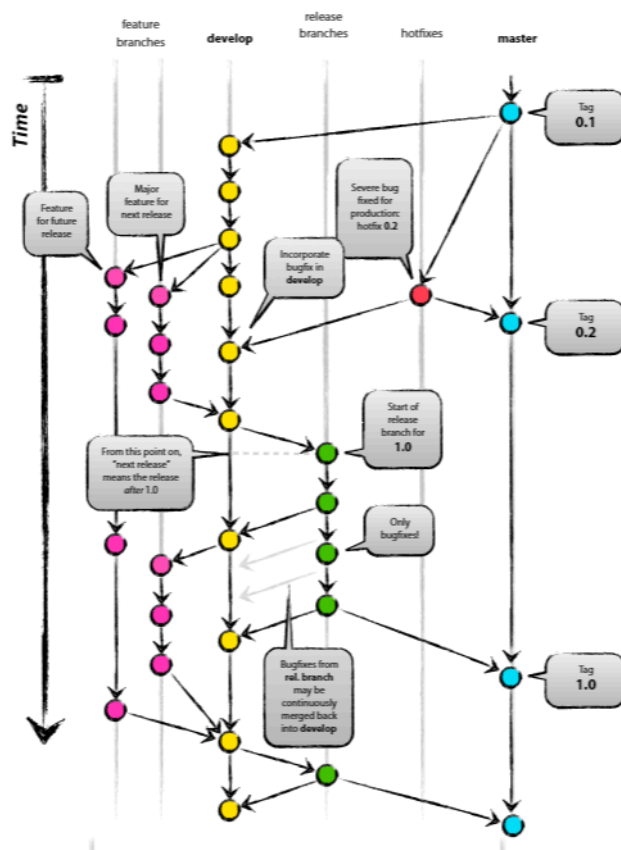
 davblamor committed 2 days ago

Fix a1.6c

 carmarbar9 committed 5 days ago

5. Estructura de los repositorios y ramas predeterminadas

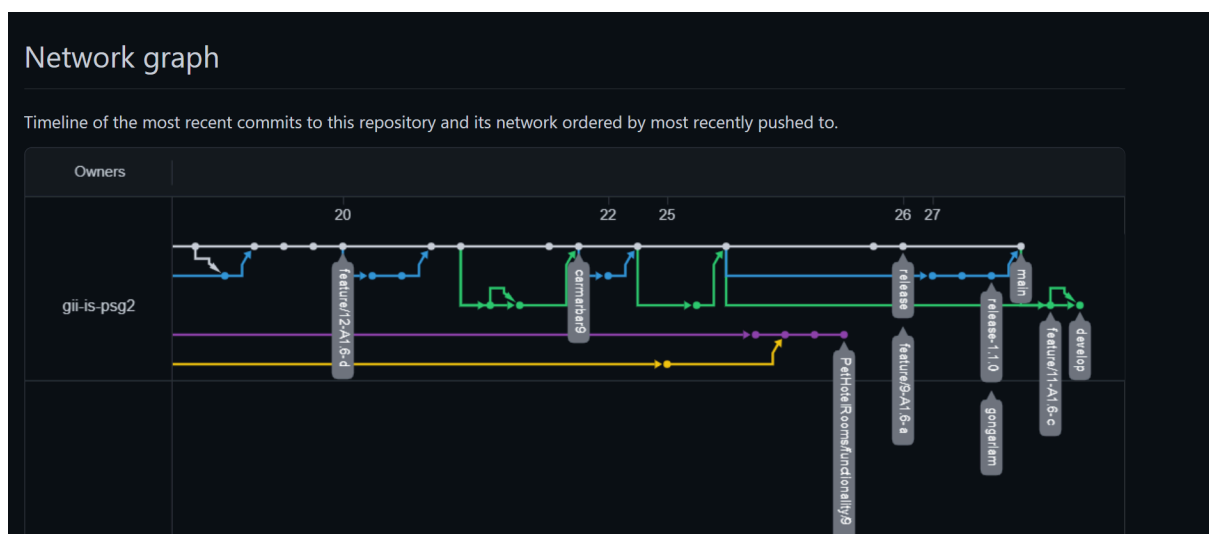
El modelo planteado en la asignatura es el siguiente:



Para cada tarea o funcionalidad que se vaya a implementar, crearemos una rama(las llamadas feature branches). Y una vez veamos que la funcionalidad implementada funciona perfectamente, la subimos a la rama develop.



Documentación Configuration Report



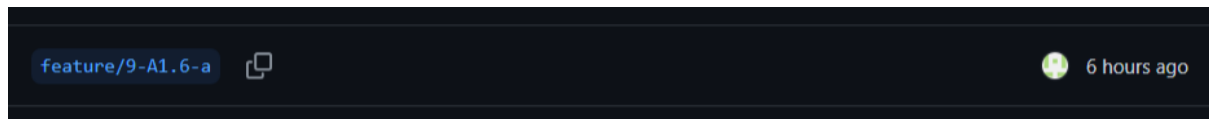
	PSG2 Documentación de la entrega Milestone 01 Documentación Configuration Report
---	--

6. Estrategia de ramificación, basada en Git Flow e incluyendo revisiones por pares

6.1. Cómo desarrollar ramas de características

Las ramas features son ramificaciones de la rama principal “develop”, la cual de por sí se ramifica de “main”. Estas features se utilizarán para la implementación de funcionalidades individuales que luego se hará un merge con develop y que no deben crear problemas de compilación ni conflictos varios.

Las ramas tendrán como nombre “feature/{idIssue}-{idProductBacklog}”. Ejemplo:
feature/9-A1.6-a:



En definitiva, una vez hecho el merge con la rama “develop” se obtendrá una versión que funciona pero que aún no tiene una funcionalidad completa que entregar al cliente.

6.2. Cómo preparar lanzamientos

Dependiendo de del tipo de versión/incremento que se vaya a realizar:

- Se considerará que se puede realizar una nueva versión mayor cuando cambiemos de Sprint, es decir cuando todas las tareas del Sprint Backlog estén hechas.
- Se considerará que se puede realizar una nueva versión menor cuando todas las subtareas de una tarea concreta estén hechas. Estas son las tareas de código que tengan un identificador de tarea igual en el Product Backlog (un ejemplo de esto serían las 2.5x, donde x es la letra que identifica a la tarea).
- Se considerará que se puede realizar un parche cuando se detecte algún error dentro del código y pueda ser arreglado.

	PSG2 Documentación de la entrega Milestone 01 Documentación Configuration Report
---	--

6.3 Cómo solucionar errores en producción

Cuando se detecte un problema, lo solucionará el creador del código que dé fallo, si fuese posible.

Si el error se detecta durante la preparación de una versión, el parche será realizado en la rama de “release”. El merge se haría a las ramas “main” y “release”.

Si el error se detecta para una versión final anterior en la rama “main”, el parche será realizado en la rama de “hotfix”. El merge se haría a las ramas “main” y “develop”.

7. Políticas de versionado

La política de versionado se centra en definir una etiqueta indicando una versión estable del proyecto. En nuestro caso se seguirá la siguiente estructura **v.x.y.z**:

- **x**: indica que es una versión con cambios mayores, siendo la versión alfa del proyecto la 1.0.
- **y**: indica que el proyecto se han realizado cambios menores. Estos cambios pueden representar la finalización de conjuntos de tareas de código estables (un ejemplo serían todas las tareas de código con el prefijo A1.8).
- **z**: indica la versión de parche o que han habido cambios en la rama hotfix para solucionar algún error.

Para clarificación, se presentan los siguientes ejemplos:

- **v.1.0.0** representará la primera release del proyecto.
- **v.1.1.0** representará que se ha realizado un conjunto de tareas.
- **v.2.1.2** representará la segunda release, con un conjunto de tareas ya implementadas y 2 parches realizados.

Hemos elegido la estructura **v.x.y.z** ya que creemos que se adecua bastante a nuestro proyecto, y nos resulta bastante familiar al haberla estudiado en teoría.

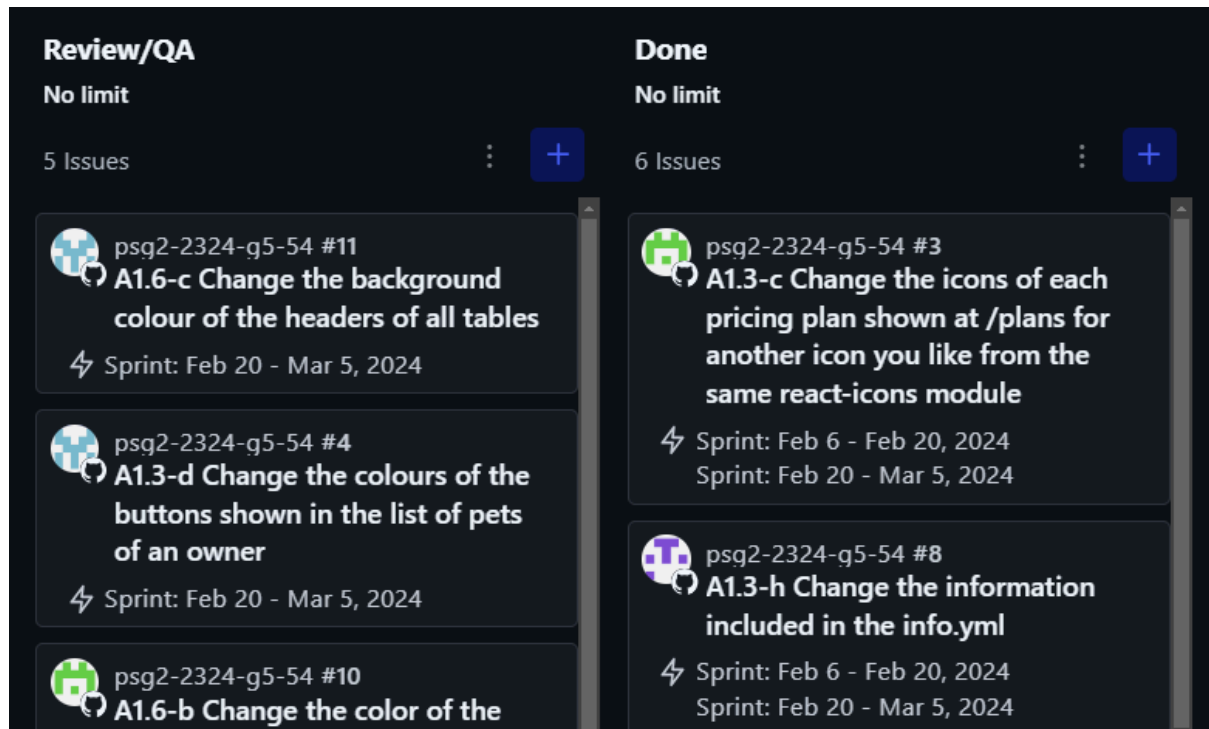
8. Definición de “hecho”

-Tareas de código: Una tarea de código se denominará como hecha en el momento en que funcione, añada la funcionalidad descrita en la tarea y no comprometa al sistema.

Además, la tarea debe estar dentro de la columna Done en el tablero del proyecto de GitHub, para llegar a este estado previamente debe ser supervisada por todos los miembros del grupo distinto al individuo que ha realizado la tarea (la tarea se encontrará previamente en la columna Review).

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

Ejemplo de tareas marcada cómo Review y cómo Done:



-Tareas de documentación: Una tarea de documentación se denominará como hecha cuando sea revisada por todo el grupo y en consenso se tome como terminada, para ello el documento debe contener todos los puntos y características proporcionados en el Product Backlog, además de no contener ninguna falta de ortografía y estar redactado de manera clara y concisa. Esta definición aplica solo a las tareas de documentación realizadas en grupo. Las tareas de documentación individuales seguirán la definición de hecho del redactor.

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

9. Definición de “Coordinación”

El equipo se organiza alrededor de un scrum manager y el resto de integrantes denominados como developers. Se ha recopilado y desglosado las actividades a realizar en sus respectivas sub-tareas para repartir y equilibrar mejor la carga de trabajo.

Para las tareas de código nos hemos organizado con las ramas que hemos comentado anteriormente lo cual nos permite trabajar de manera coordinada y eficiente.

Para las tareas de documentación hemos usado documentos que nos permite la escritura simultánea y coordinada de todos los documentos que se van a generar durante este sprint.

Desglose de tareas:

Índice	Historia
A1.3-a	Add your information to the developers' section in pom.xml
A1.3-b	Change the project information in pom.xml
A1.3-c	Change the icons of each pricing plan
A1.3-d	Change the colors of the buttons shown in the list of pets
A1.3-e	Change the message of the home page
A1.3-f	Change the message of the home page
A1.3-g	Change the background color of the table header shown on the Consultations list page while logged in as an owner
A1.3-h	Change the information included in the info.yml file
A1.4	Create a technical report
A1.5	Create a technical report that describes the configuration management methodology followed by the group
A1.6-a	Create a new functionality for Clinic Owners
A1.6-b	Change the color of the gradients shown in each card of the /plans page
A1.6-c	Change the background color of the headers of all tables

Como selección de puntos de historia en las tareas, hemos usado la estrategia de Poker para evaluar la dificultad y el tiempo que nos podría llevar en esas tareas.

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

10. Cómo se gestionan los documentos generados durante el proyecto

Los documentos se gestionan dentro de una carpeta '**docs**' en la rama principal del proyecto. Esta carpeta consta de otras carpetas dentro para hacer referencia a los respectivos Sprints, 'MilestoneX', siendo X el número que hace referencia a cada Sprint.

[psg2-2324-g5-54 / docs /](#) 

Add file ▾ ⋮

 GonxKZ Adición de Sprint Review	5a67106 · 10 hours ago	History
---	------------------------	---------

Name	Last commit message	Last commit date
 ..		
 Milestone1	Adición de Sprint Review	10 hours ago
 Milestone2	Modificación de Documentos	10 hours ago
 Milestone3	Modificación de Documentos	10 hours ago
 Milestone4	Modificación de Documentos	10 hours ago

Dentro de cada carpeta de Sprint, se tendrán en cuenta los siguientes documentos:

- Informes de Scrum Diarios
- Reuniones
- Retrospectiva del Sprint
- Revisión del Sprint
- Planificación del Sprint

[psg2-2324-g5-54 / docs / Milestone1 /](#) 

Add file ▾ ⋮

 GonxKZ Adición de Sprint Review	5a67106 · 10 hours ago	History
---	------------------------	---------

Name	Last commit message	Last commit date
 ..		
 1.4 Technical Report.pdf	Modificación de Documentos	10 hours ago
 1.5 Configuration Report.pdf	Modificación de Documentos	10 hours ago
 13-02-2024.docx.pdf	Modificación de Documentos	10 hours ago
 20-02-2024.docx.pdf	Modificación de Documentos	10 hours ago
 Sprint Planning 1.docx.pdf	Modificación de Documentos	10 hours ago
 Sprint Retrospective 1.docx.pdf	Modificación de Documentos	10 hours ago
 Sprint Review 1.docx.pdf	Adición de Sprint Review	10 hours ago

	PSG2 Documentación de la entrega Milestone 01
	Documentación Configuration Report

11. Objetivos y ámbito de CMDB

Hemos utilizado la funcionalidad de CMDB (Base de Datos de Configuración) en iTop con los siguientes objetivos:

- Gestionar servicios de TI, activos y operaciones de manera estructurada y eficiente.
- Gestionar incidentes y problemas.
- Servir apoyo a la definición de lo hecho
- Control y seguimiento de Cambios

En nuestro caso contamos hemos añadido varios elementos de configuración, contamos con:

- 6 ordenadores que cada uno corresponde al portátil personal de cada uno de los integrantes del grupo.
- 3 PC Software's instalados en los ordenadores comentados anteriormente
- 1 Application Solution
- 2 Serveridores
- 1 Periférico

iTop PC Export								Page 1
id (Primary Key)	Organization	Status	Brand	Model	Type	RAM	CPU	
David's PC	PSG2-2324-G5-54	implementation	HP Pavilion	undefined	laptop	16Gb	i7	
Gonzalo	PSG2-2324-G5-54	production	Acer	Nitro 5	laptop	16GB DDR4	Intel i7-10750H	
juanunsan2	PSG2-2324-G5-54	production	Asus	TUF GAMING F15	laptop	16GB	11th Gen Intel(R) Core(TM) i7-11370H @ 3.30GHz 3.30 GHz	
LAPTOP(Yao)	PSG2-2324-G5-54	production	HP OMEN	15-en1xxx	laptop	16GB	Intel(R) Core(TM) i7-10870H CPU @ 2.20GHz 2.21 GHz	
Portátil (carmarbar9)	PSG2-2324-G5-54	Undefined	DELL	XPS 13 9370	laptop	16GB	Intel(R) Core(TM) i7-8550U CPU	
Portátil (lidjmsor)	PSG2-2324-G5-54	production	MSI	Modern 14 B11MOU	laptop	32,0GB	11th Gen Intel Core i7	

iTop PC Software Export			Page 1
Software	Organization	CI sub-class	
Node.js v20.11.0	PSG2-2324-G5-54	PC Software	
Visual Studio Code 1.86.2	PSG2-2324-G5-54	PC Software	
Zenhub Zenhub Web Extension	PSG2-2324-G5-54	PC Software	

iTop Application Solution Export				Page 1
id (Primary Key)	Organization	Move to production date	Description	
PetClinic Application	PSG2-2324-G5-54	2024-02-20	Nuestra aplicación de solución, 'PetClinic', es una aplicación web integral diseñada para simplificar el proceso de reserva de citas y mejorar la comunicación entre las clínicas veterinarias y los propietarios de mascotas. Con PetCare Connect, gestionar la salud de las mascotas se vuelve sencillo y eficiente tanto para las clínicas como para los propietarios de mascotas.	



PSG2 Documentación de la entrega Milestone 01

Documentación Configuration Report



Server Export

Page 1

id (Primary Key)	Organization	CI sub-class
Clockify for our Organization	PSG2-2324-G5-54	Server
Discord server for organization	PSG2-2324-G5-54	Server



Peripheral Export

Page 1

id (Primary Key)	Organization	CI sub-class
Monitor (juanunsan2)	PSG2-2324-G5-54	Peripheral

